

android

- [android](#)
 - [características](#)
 - [Android Studio vs. IntelliJ IDEA](#)
 - [Crear proyecto Compose](#)
 - [Ejemplo](#)
 - [preview](#)
 - [Explicación del código:](#)
 - [organizar el código](#)
 - [Diseñando la vista de nuestra app](#)
 - [Agregando características](#)
 - [Diseño horizontal](#)
 - [Descarga](#)

Android Studio es el entorno de desarrollo integrado oficial para crear aplicaciones Android . Reúne todas las herramientas que un desarrollador de apps necesita en un solo lugar. Android Studio está basado en el potente software IntelliJ IDEA de JetBrains, pero ha sido modificado y mejorado por Google específicamente para el desarrollo de Android . Fue anunciado en 2013 y lanzado oficialmente en 2014, reemplazando a Eclipse como el IDE oficial para este propósito

características

- Editor Inteligente: Soporta de forma nativa Kotlin, Java y C++ . Ofrece autocompletado de código, detección de errores en tiempo real y refactorización (renombrar variables, mover métodos, etc.) de forma segura y automática.
- Sistema de Compilación Flexible (Gradle): El corazón de la construcción de proyectos. Gradle te permite crear diferentes "variantes" de tu app desde un solo proyecto, por ejemplo, una versión gratuita con anuncios y una versión de pago sin ellos, personalizando cada aspecto del proceso de compilación . Los archivos de configuración se llaman `build.gradle.kts` (usando Kotlin, que es el recomendado) o `build.gradle` (con Groovy) .
- Emulador Rápido y Potente: Olvídate de necesitar un teléfono físico para cada prueba. El emulador de Android Studio te permite crear dispositivos virtuales de prácticamente cualquier modelo, tamaño de pantalla y versión de Android (teléfonos, tablets, relojes, TVs) para probar tus apps directamente en tu computadora .

- **Diseño de Interfaces (Jetpack Compose):** Para crear las pantallas de tu app, Android Studio ofrece herramientas visuales que te permiten arrastrar y soltar componentes (botones, textos, imágenes) y ver los cambios reflejados en tiempo real, sin necesidad de compilar todo el proyecto de nuevo .
- **Perfiladores y Herramientas de Depuración:** Incluye analizadores en tiempo real del uso de CPU, memoria, red y batería de tu app. Puedes pausar la ejecución, inspeccionar el valor de las variables, y encontrar "cuellos de botella" o fugas de memoria que afectarían el rendimiento.

Android Studio vs. IntelliJ IDEA

Característica	Android Studio	IntelliJ IDEA (Community)
Enfoque Principal	Desarrollo de apps para Android (móviles, tablets, wearables, TV) .	Desarrollo general para JVM (Kotlin, Java, backend, aplicaciones de escritorio).
Creador / Soporte	Desarrollado por Google (sobre la base de IntelliJ) .	Desarrollado por JetBrains (los creadores de Kotlin).
Herramientas Exclusivas	Emulador de Android, editores de diseño de layouts (XML y Compose), herramientas específicas de Google Play (firmado, análisis de APK) .	Soporte superior para desarrollo web, frameworks de backend (Spring, Ktor), bases de datos y herramientas de diagramación UML.

¿Cuál elegir? Es la elección obligatoria si quieres crear apps para Android . Es la mejor opción si quieres aprender Kotlin para backend, scripts de consola o aplicaciones de escritorio multiplataforma . En resumen, todo lo que puedes hacer en Android Studio está basado en IntelliJ IDEA . Son "hermanos", pero cada uno está especializado en un tipo de proyecto.

Crear proyecto Compose

Jetpack Compose es el kit de herramientas moderno de Google para crear interfaces de usuario (UI) en Android usando solo código Kotlin, sin necesidad de XML. Es un cambio radical respecto a la forma tradicional de hacer interfaces en Android.

Creación de un Proyecto en Android Studio con Compose.

1. Al abrir Android Studio, selecciona **New Project** .

2. Para que elijas una plantilla para tu primera pantalla. Se abrirá una ventana con varias opciones. Aquí debes seleccionar la pestaña "Phone and Tablet" y luego elegir la plantilla **Empty Activity** . Esta plantilla es la más recomendada para empezar con Compose. Crea un proyecto simple con una sola pantalla y todo lo necesario para que veas tu primer "Hola Mundo" funcionando
3. Configura tu proyecto: En la siguiente pantalla, completa los detalles de tu app. Los más importantes son:
 - *Name*: El nombre de tu aplicación (por ejemplo, "MiPrimeraAppCompose").
 - *Package name*: El identificador único de tu app (suele ser algo como com.tunombre.miprimeraappcompose).
 - *Save location*: La carpeta donde se guardará el proyecto.
 - *Language*: Asegúrate de que esté seleccionado Kotlin. Es el único lenguaje compatible con Compose.
 - *Minimum SDK*: Selecciona API 24: Android 7.0 (Nougat) o una versión posterior. Esto define la versión más antigua de Android en la que funcionará tu app.
 - *Use Jetpack Compose UI*: Asegúrate de que esta casilla esté marcada. ¡Esto es crucial! Activa todas las configuraciones necesarias para usar Compose.
4. Haz clic en **Finish**. *Android Studio* generará toda la estructura de carpetas y archivos y sincronizará el proyecto con Gradle automáticamente.

Breve descripción de los archivos:

Gradle Scripts es un archivo de configuración, si abres **build.gradle.kts** veras, en *plugings* los módulos importados, en *android* la descripción de proyecto.

app/kotlin+java/com.sunombre.suproyecto/ui.theme contiene los estilos de compose.

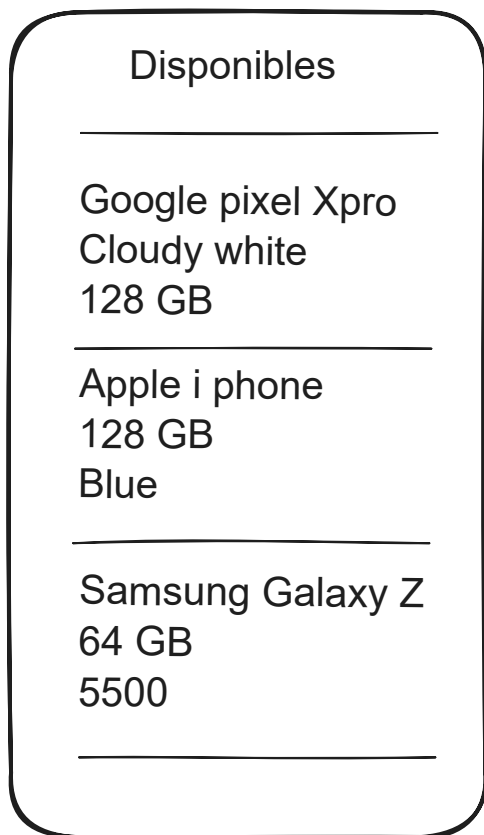
app/kotlin+java/MainActivity es el código principal.

Ejemplo

Crear una aplicación móvil con *Jetpack compose*. *Jetpack Compose* es el kit de herramientas moderno de Android para construir interfaces de usuario (UI) de forma declarativa utilizando código Kotlin. Fue lanzado por Google para reemplazar el sistema tradicional de vistas basado en XML (como LinearLayout, TextView, Button, etc.). Ejemplo: en lugar de decir cómo construir la UI paso a paso (imperativo), con Compose declaras cómo debe verse la UI según el estado actual de los datos. Ejemplo:

- Antes `findViewById<TextView>(R.id.texto).text = "Hola"`
- Ahora: `Text("Hola")`

La Interfaz de la app será la siguiente:



Inicio

Si aunnn no has creado un proyecto Compose, sigue los pasos 1 y 2, en caso contrario ve al paso 3.

1. Crea un nuevo proyecto

- file/new project/
- elige la plantilla `empty activity`
- elige nombre del **Package name** por ejemplo `myapp-jetp-1`
- en **Build configuration language** elegir **kotlin**
- en **minimun sdk** versión mínima de **Android** donde correra tu app.
- da clic en `next` .

2. revisa el archivo `/src/MainActivity.kt` es el equivalente al `main()` en lenguaje C.

3. Ubica una api fake: [Api fake](#)

4. elige [all](#)

5. crea una clase para almacenar los datos:

1. colócate en el paquete principal: **com.tunombre.suproyecto** haz clic derecho/new/(kotlin class/file)
2. elije **data class**
3. ponle como nombre *Device*
4. ponle las propiedades:

```
data class Device(  
    val id: Long,  
    val name: String,  
    val data: Specs?  
)
```

5. observa que data hace referencia a Specs (especificaciones del objeto como color y capacidad [observa los datos]), por tanto necesitamos crear un nuevo *class data* que llamaremos *Specs*, construyela creando un archivo *Specs* similar a como lo hiciste con el *class data Device* ponle los campos:

```
data class Specs(  
    val color: String?,  
    val capacity: String?  
)
```

Nota Una **Data Class** es una clase especial en Kotlin diseñada exclusivamente para almacenar datos. Automáticamente genera código útil como `equals()`, `hashCode()`, `toString()`, `copy()`, y los métodos de componente para deestructuración.

6. Revisa el código

Observa en el archivo `Main.activity` que tenemos 3 cosas:

- Una actividad (la clase `MainActivity`)
- la función composable para la vista (`Greeting`)
- la previsualización (`GreetingPreview`)

La clase hereda de `ComponentActivity`. Una Activity es un componente fundamental de una aplicación Android que representa una pantalla con la que el usuario puede interactuar.

La función `fun onCreate()` se ejecuta al crear la actividad, dentro puede ver `setContent {}` que es una sentencia del Jetpack Compose que añade la interfaz gráfica a la actividad.

`DeviceTheme{}` define el tema, es tomado de Material design.

`scaffold{}` es el contenedor principal de la interfaz.

dentro se llama a la función `Greeting(name = "Android", modifier = Modifier.padding(innerPadding))`. Las vistas en Jetpack compose se construyen a partir de funciones kotlin marcadas con `@Composable` y se deben escribir en Pascal Case.

El parámetro `modifier`: Un Modifier es un objeto que modifica el comportamiento y apariencia de un componente de Compose. Puedes encadenar múltiples modificaciones.

Ejemplo de uso de modifier encadenado:

```
Greeting(  
    name = "María",  
    modifier = Modifier  
        .fillMaxWidth()  
        .padding(horizontal = 20.dp)  
        .clip(RoundedCornerShape(8.dp))  
        .border(2.dp, Color.Blue)  
) // El saludo ocupará todo el ancho, con bordes redondeados azules
```

en la función `Greeting` tenemos un componente:

```
Text(  
    text = "Hello $name!",  
    modifier = modifier  
)
```

Nota Un componente de Android es un bloque de construcción fundamental de una aplicación Android. Cada componente tiene un propósito específico y un ciclo de vida propio, y pueden activarse de diferentes maneras (por el usuario, el sistema u otros componentes).

preview

Para empezar una previsualización, selecciona en la parte superior derecha el ícono *split*, esto habilita una ventana de previsualización.

En tu código del archivo `MainActivity.kt` observa:

```

@Preview(showBackground = true, showSystemUi = true)
@Composable
fun GreetingPreview() {
    MyAppjetp1Theme {
        Greeting("Android", modifier = Modifier.padding(top = 24.dp))
    }
}

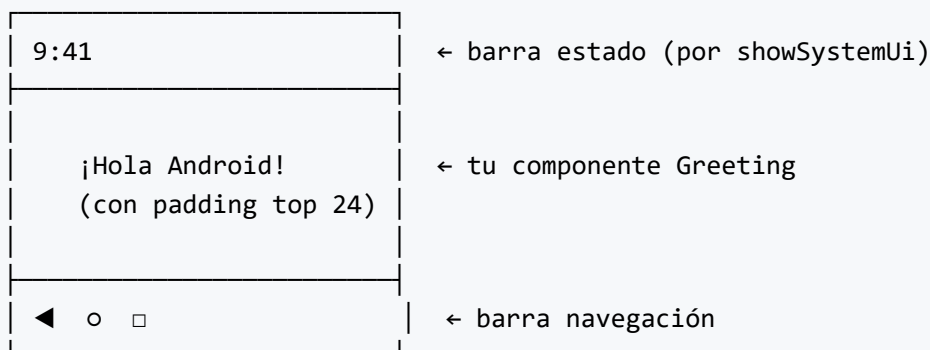
```

Es una vista previa en tiempo real de tu interfaz de usuario sin necesidad de ejecutar la app en un emulador o dispositivo físico. Es una característica de Jetpack Compose (el toolkit moderno de UI para Android). En este caso previsualiza el componente *Greeting*

Desglose de cada parte:

Elemento	Función
@Preview	Le dice a Android Studio / IntelliJ que esta función es una "vista previa" que debe mostrarse en una ventana lateral
showBackground = true	Muestra un fondo detrás de tu UI (útil para ver bordes blancos sobre fondo oscuro)
showSystemUi = true	Muestra la barra de estado y la barra de navegación del teléfono (hora, batería, botones virtuales)
@Composable	Indica que esta función define una parte de la UI (la unidad básica de Jetpack Compose)
Myappjetp1Theme	Aplica el tema visual de tu app (colores, tipografías, formas)
Greeting(...)	El componente real que estás previsualizando

Resultado:



Explicación del código:

```
fun GreetingPreview() {}
```

- Es una función normal de Kotlin
- El nombre puede ser cualquiera, pero por convención se usa [NombreComponente]Preview
- No recibe parámetros ni devuelve nada

```
Myappjetp1Theme {}
```

- Aplica el tema visual de tu app (colores, tipografías, formas, espaciados)
- Sin esto, verías los colores "por defecto" del sistema, no los que diseñaste para tu app
- El nombre Myappjetp1Theme suele venir del archivo Theme.kt que genera Android Studio automáticamente

```
Greeting("Android", modifier = Modifier.padding(top = 24.dp))
```

- Llama a tu componente Greeting

organizar el código

En el archivo `MainActivity.kt` dejaremos solo la clase y la vista y la previsualización las llevaremos a otro archivo.

En la carpeta donde esta el *MainActivity* creamos un nuevo archivo llamado *MainScreen*, dando clic derecho en el nombre de la carpeta/new file/(kotlin class/file)/ file, ponga el nombre *MainScreen*

mueva ahi el código:


```

@Composable
fun Greeting(name: String, modifier: Modifier = Modifier) {
    Text(
        text = "Hello $name!",
        modifier = modifier
    )
}

@Preview(showBackground = true, showSystemUi = true)
@Composable
fun GreetingPreview() {
    MyAppjetp1Theme {
        Greeting("Android", modifier = Modifier.padding(top = 24.dp))
    }
}

```

Diseñando la vista de nuestra app

Crearemos la vista de un solo dispositivo.

1. crear un nuevo archivo kotlin llamado *DeviceItem*
2. agregue el código:

```

@Composable
fun DeviceView(device: Device){
    Text(text = device.name)
}

```

```

@Preview(showBackground = true) @Composable fun DeviceItemPreview(){ MyAppjetp1Theme
{ DeviceView(device = Device(id = 1, name= "Nexus", Specs(color = "White", capacity =
"64gb")))) } }

```

Y muestre su previsualización

Agregando características

Modifique la siguiente función:

```
```kotlin
@Composable
fun DeviceView(device: Device){
 Column {
 Text(text = device.name)
 Text(text = device.data?.color ?: "-")
 Text(text = device.data?.capacity ?: "-")
 }
}
```

Observe la sentencia: `column{}` indica que los elementos se ubicarán en el lienzo en forma de columna.

La expresión: `device.data?.color ?: "-"`

- Si `device.data` existe → sigue adelante a `.color`
- Si `device.data` es null → todo esto se convierte en null

Operador elvis: `?:` Significado: Si lo de la izquierda es null, usa lo de la derecha.

## Diseño horizontal

---

clase del jueves.

## Descarga

---

[Descargar Android studio](#)

Nota Requerimientos: memoria 8mb, SO recomendado 64 bits, procesador intel core 2da generacion+. Disco 16 gb+, resolución 1920\*1080.